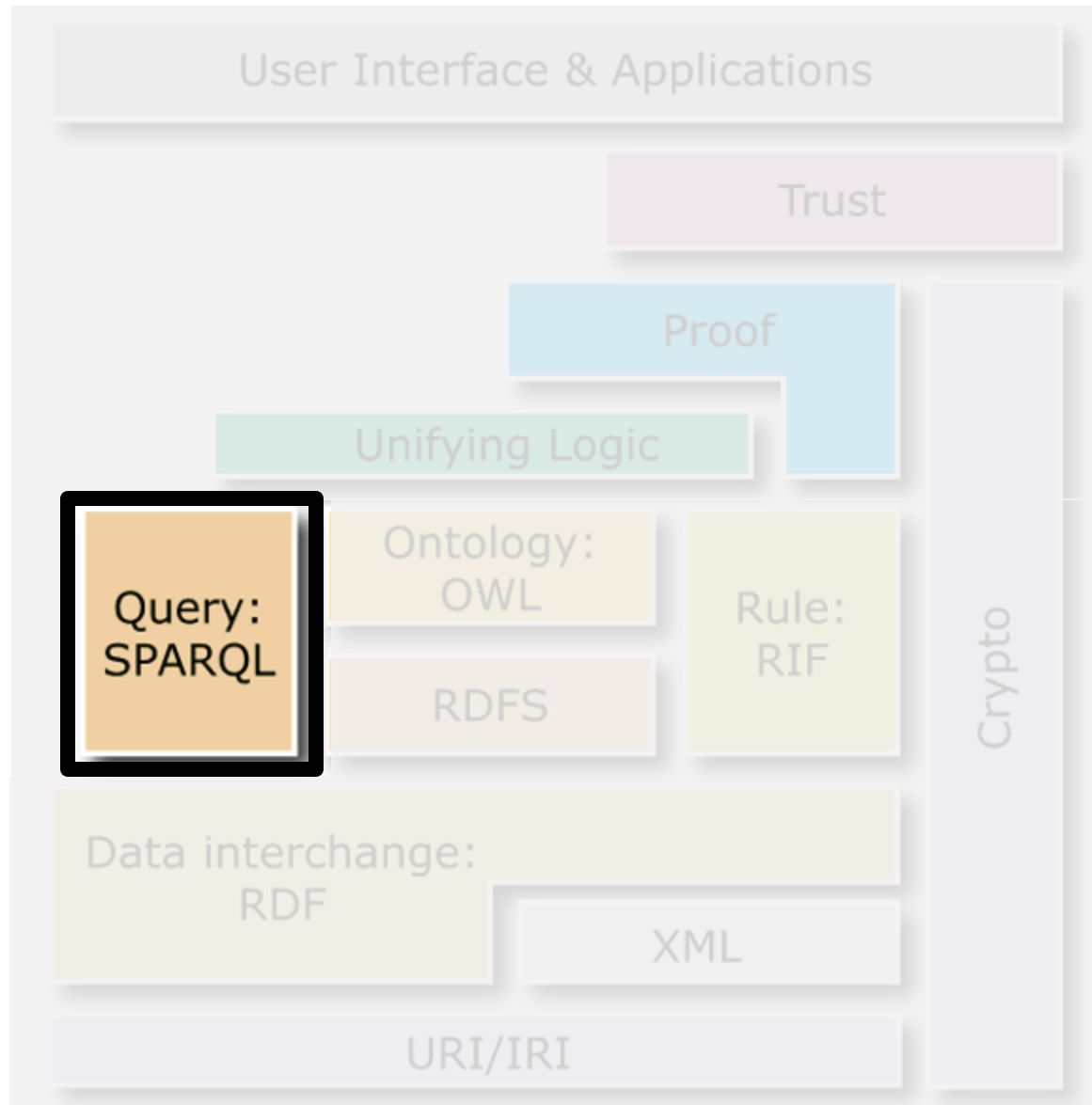


Web & Data Science

Querying RDF with SPARQL

25/09/2025

Query: SPARQL



What is SPARQL?

- SPARQL
 - is the query language of the Semantic Web
 - stays for **SPARQL Protocol** and **RDF Query Language**

- A Query Language ...

Find drugs in Dbpedia

```
PREFIX dbpedia-owl: <http://dbpedia.org/ontology/>
SELECT ?s WHERE {
    ?s a dbpedia-owl:Drug .
}
```

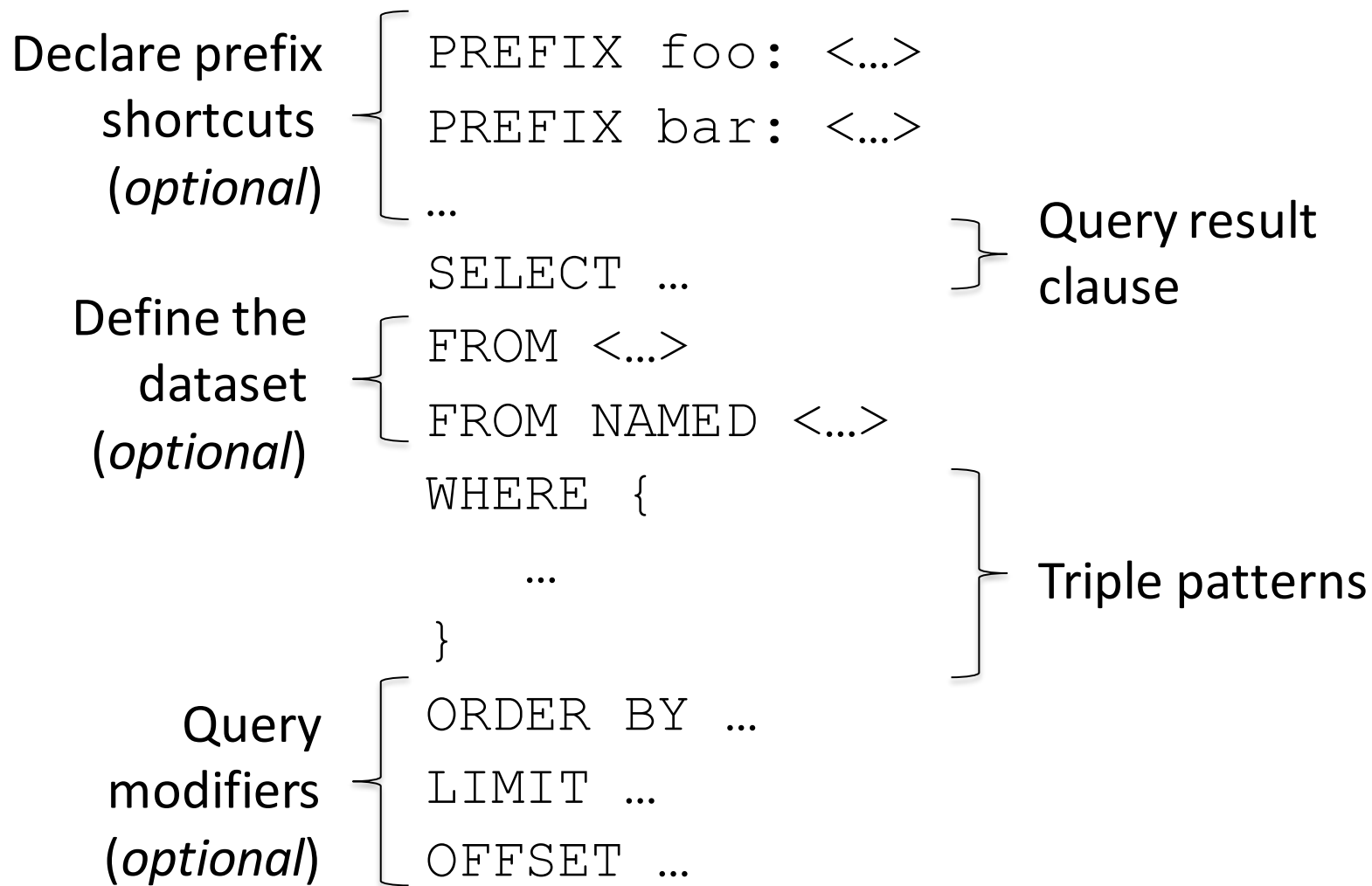
- ... and a Protocol

```
http://factforge.net/sparql?name=&infer=false&sameAs=false
&query=PREFIX%20dbpedia-owl%3A%20%3Chttp%3A%2F%2Fdbpedia.org
%2Fontology%2F%3E%0ASELECT%20%3Fs%20WHERE%20%7B%20%0A%09%3Fs
%20a%20dbpedia-owl%3ADrug%20.%20%0A%7D%0A%0A%0A
```

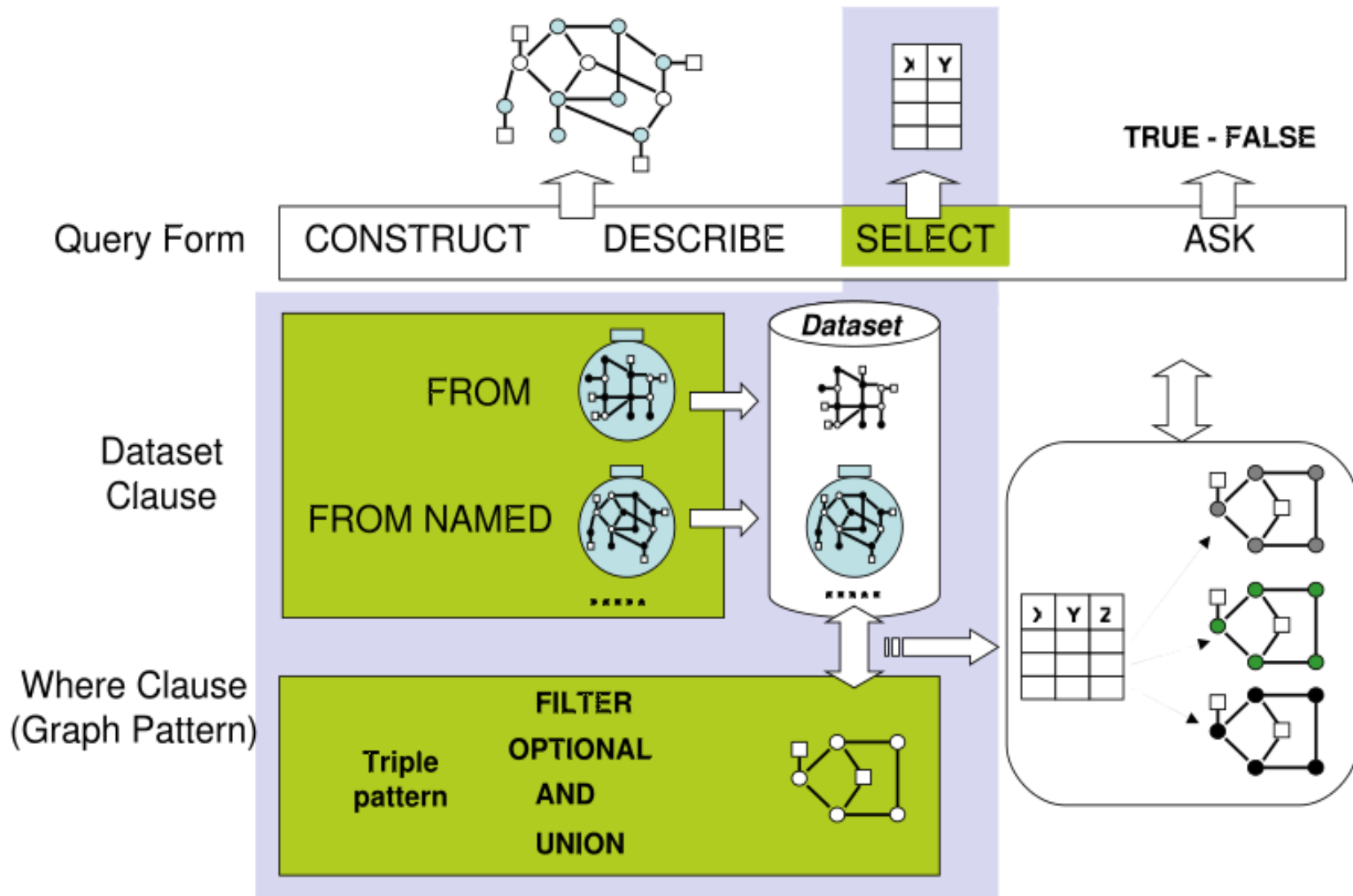
Why SPARQL?

- SPARQL let us
 - Pull values from structured and semi-structured data represented in RDF
 - Explore RDF data by querying unknown relationships
 - Perform complex joins of disparate RDF repositories in a single query
 - Transform RDF data from one vocabulary to another
 - Develop higher-level cross-platform application

Anatomy of a SPARQL query

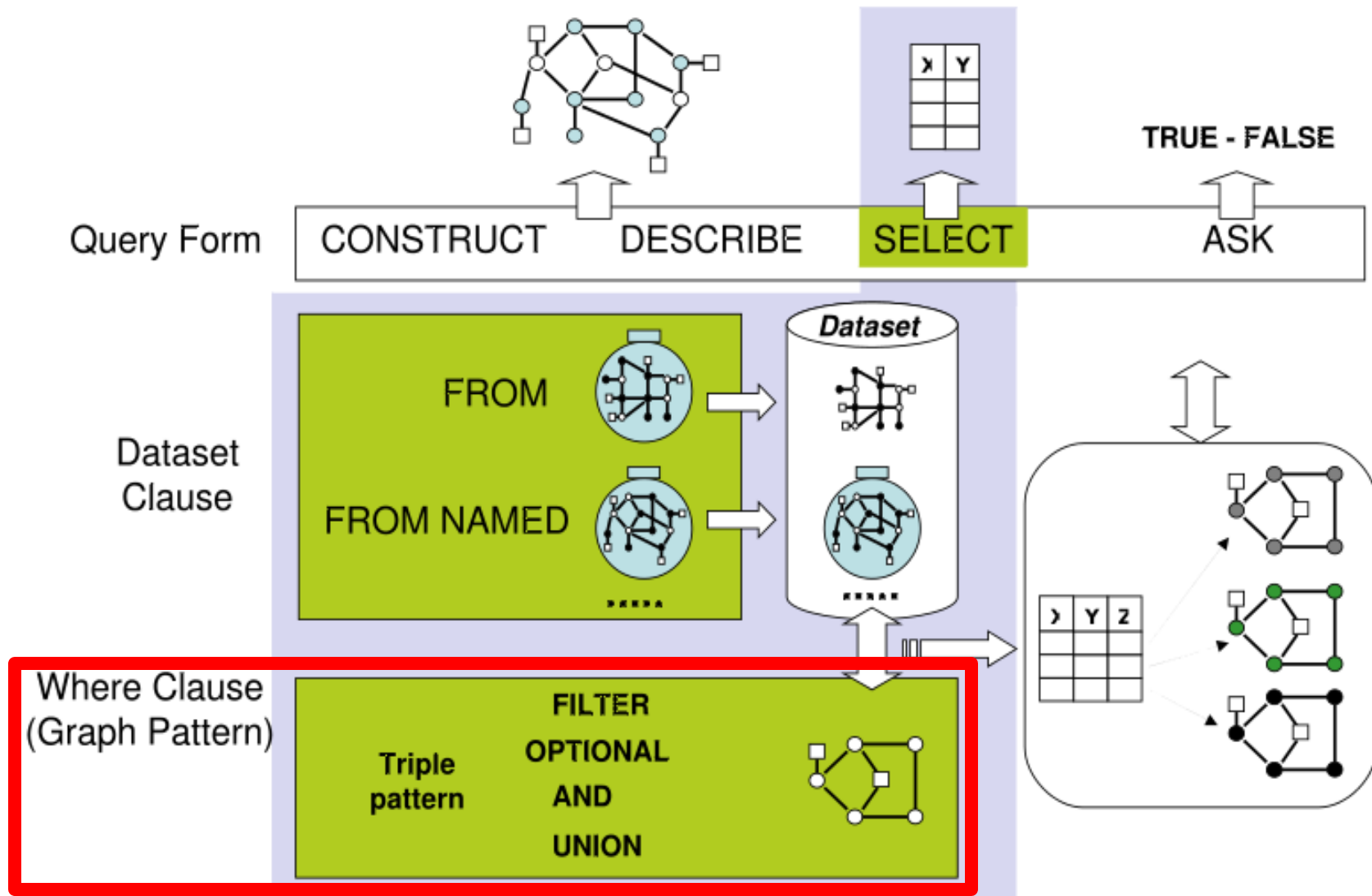


Anatomy of a SPARQL query



Source: Pérez, Arenas and Gutierrez, Chapter 1: On the Semantics of SPARQL, Semantic Web Information Management: A Model Based Perspective, Springer 2010

Anatomy of a SPARQL query



Source: Pérez, Arenas and Gutierrez, Chapter 1: On the Semantics of SPARQL, Semantic Web Information Management: A Model Based Perspective, Springer 2010

Triple Pattern Syntax

- Turtle-like: URIs, QNames, literals, convenience syntax.
- Adds variables to get basic patterns
 - ?var
 - Variable names are a subset of NCNames (no "-" or ".")
- E.g.,
 - simple
 - `?s a :Artwork .`
 - complex
 - `?s a :Artwork .`
 - `?s :author ?o .`
 - `?s ?p :Michelangelo .`
- Adds
 - OPTIONAL to cope with semi-structured nature of RDF
 - FILTER to select solution according to some criteria
 - UNION operator to get complex patterns

Writing a Simple Query

■ Query

```
PREFIX : <http://example.org/ontology/>
```

```
SELECT ?s
```

```
WHERE { ?s a :Artwork . }
```

■ Results

?s
http://example.org/ontology/Sistine_Chapel
http://example.org/ontology/David
http://example.org/ontology/Last_Supper
http://example.org/ontology/Mona_Lisa

Writing a bit more complex query

■ Query

```
PREFIX : <http://example.org/ontology/>
```

```
SELECT ?s ?o
```

```
WHERE { ?s a :Artwork .  
        ?s :author ?o . }
```

■ Results

?s	?o
http://example.org/ontology/Sistine_Chapel	http://example.org/ontology/Michelangelo
http://example.org/ontology/David	http://example.org/ontology/Michelangelo
http://example.org/ontology/Last_Supper	http://example.org/ontology/Leonardo
http://example.org/ontology/Mona_Lisa	http://example.org/ontology/Leonardo

Writing a query that asks for the properties

- Query

```
PREFIX : <http://example.org/ontology/>
```

```
SELECT ?p
```

```
WHERE { :Leonardo ?p :Italy . }
```

- Results

?p
http://example.org/ontology/birthPlace

Basic Graph Patterns

- A basic graph pattern is a set of triple patterns, all of which **must be matched**.
- In this case *matches the graph* means find a set of bindings such that the substitution of variables for values **creates a subgraph** that is in the set of triples making up the graph.

Are you following? Let's check!



[source http://origin-www.yoursingapore.com/content/traveller/zh/browse/see-and-do/hands-on/_jcr_content/flash/image.img.png]

Put what illustrated at work

- Find the artists in the RDF dataset
(hint: the URL of artist is <http://example.org/ontology/Artist>)
- Find the artists born in Italy
(hint: we saw born few slides back)
- What links Michelangelo to Italy?

Put what illustrated at work - SOLUTION

- Find the artists in the RDF dataset
(hint: the URL of artist is <http://example.org/ontology/Artist>)

```
PREFIX : <http://example.org/ontology/>
```

```
SELECT ?s
```

```
WHERE { ?s a :Artist . }
```

- Results

?s
http://example.org/ontology/Michelangelo
http://example.org/ontology/Leonardo

Put what illustrated at work - SOLUTION

- Find the artists born in Italy
(hint: we saw born few slides back)

```
PREFIX : <http://example.org/ontology/>
```

```
SELECT ?s
```

```
WHERE { ?s a :Artist .  
        ?s :birthPlace :Italy . }
```

Results

?s

http://example.org/ontology/Michelangelo

http://example.org/ontology/Leonardo

Put what illustrated at work - SOLUTION

- What links Michelangelo to Italy?

```
PREFIX : <http://example.org/ontology/>
```

```
SELECT ?p
```

```
WHERE { :Michelangelo ?p :Italy . }
```

- Results

?p
http://example.org/ontology/deathPlace
http://example.org/ontology/birthPlace

- Query

```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?s
WHERE { ?s rdfs:label "Mona Lisa" . }
```

- Results for the query:

0 results!!!

- But the resource exists in dbpedia

http://example.org/ontology/Mona_Lisa

- Explanation

- The *rdfs:label* resource has a language tag
- "Mona Lisa" != "Mona Lisa"@en

■ Query

```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?s
WHERE { ?s rdfs:label "Mona Lisa"@en . }
```

■ Results

?s

http://example.org/ontology/Mona_Lisa

Matching RDF literals – numerical values

- As in the case of language tags, if the literals are typed (e.g., "1452-04-15"^^**xsd:date**), they do not match if they are not given explicitly.

- Query

```
PREFIX :      <http://example.org/ontology/>
PREFIX xsd:   <http://www.w3.org/2001/XMLSchema#>

SELECT ?s
WHERE { ?s :birthDate "1452-04-15"^^xsd:date . }
```

- Results

```
?s
```

```
http://example.org/ontology/Leonardo
```

RDF Term Constraints

- SPARQL allows restricting solutions by applying the FILTER clause.
- An RDF term bound to a variable appears in the results if the FILTER expression, applied to the term, evaluates to TRUE.

- Query

```
PREFIX :      <http://example.org/ontology/>
PREFIX xsd:  <http://www.w3.org/2001/XMLSchema#>

SELECT ?s ?o
WHERE { ?s :birthDate ?o .
        FILTER (?o > "1450-01-01"^^xsd:date &&
                 ?o < "1499-12-31"^^xsd:date )
      }
```

- Results

?s	?o
http://example.org/ontology/Michelangelo	"1475-03-06"^^<http://www.w3.org/2001/XMLSchema#date>
http://example.org/ontology/Leonardo	"1452-04-15"^^<http://www.w3.org/2001/XMLSchema#date>

RDF Term Constraints – regex

- SPARQL FILTERs allows also restricting values of strings using the `regex()`
- We use `xsd:string(?o)` to perform a type casting from `xsd:date` to `xsd:string`

- Query

```
PREFIX :      <http://example.org/ontology/>
PREFIX xsd:  <http://www.w3.org/2001/XMLSchema#>

SELECT ?s ?o
WHERE { ?s :birthDate ?o .
        FILTER (regex(xsd:string(?o), "1452", "i"))
      }
```

- Results

?s	?o
http://example.org/ontology/Leonardo	"1452-04-15"^^<http://www.w3.org/2001/XMLSchema#date>

Value Tests

- Notation for value comparison: $<$, $>$, $=$, $\leq$, $\geq$ and $\neq$
- Test functions: `isIRI`, `isLITERAL`, ...
- Accessing accessories: `LANG`, `DATATYPE`
- Logic operators: `||` and `&&`
- Comparing strings: `REGEX`, `langMatches`
- Constructor functions: `bool`, `dbl`, `flt`, `dec`, `int`, `dT`, `str`, `IRI`
- Extensible Value Testing
 - E.g., `FILTER ( aGeo:distance(?axLoc, ?ayLoc, ?bxLoc, ?byLoc) < 10 ) .`
 - (see <http://www.w3.org/TR/rdf-sparql-query/#extensionFunctions>)

Are you following? Let's check!



[source http://origin-www.yoursingapore.com/content/traveller/zh/browse/see-and-do/hands-on/_jcr_content/flash/image.img.png]

Put what illustrated at work

- Find the artworks painted after the year 1500
(hint: <http://example.org/ontology/year>)
- Find the Italian names of the paintings
(hint: `if ?x = "Mona Lisa"@en, then LANG(?x) → "en")`)

Put what illustrated at work - SOLUTION

- Find the artworks painted after the year 1500
(hint: <http://example.org/ontology/year>)

```
PREFIX : <http://example.org/ontology/>
```

```
SELECT ?s
```

```
WHERE { ?s a :Artwork .  
        ?s :year ?yr .  
        FILTER (?yr >= 1500) . }
```

Put what illustrated at work - **SOLUTION**

- Find the Italian names of the paintings
(hint: if `?x = "Mona Lisa"@en`, **then** `LANG(?x) → "en"`)

```
PREFIX : <http://example.org/ontology/>
```

```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
```

```
SELECT ?s ?o
```

```
WHERE { ?s a :Painting .
```

```
        ?s rdfs:label ?o.
```

```
        FILTER(LANG(?o) = "it") }
```

More Sophisticated Graph Patterns

- RDF is "semi structured" and has no integrity constraints
- SPARQL addresses this issue with
 - **Group patterns** match if all subpatterns match and all constraints are satisfied
 - In SPARQL syntax, groups are { ... }
 - **OPTIONAL graph patterns** accommodate the need to add information to a result but without the query failing just because some information is missing.
 - In SPARQL syntax, OPTIONAL { ... }
 - **UNION graph patterns** allows to match alternatives
 - In SPARQL syntax, { ... } UNION { ... }

■ Query

```
PREFIX : <http://example.org/ontology/>

SELECT ?s ?h ?w
WHERE { ?s a :Artwork.
        OPTIONAL { ?s :heightMetric ?h .
                    ?s :widthMetric ?w . }
}
```

■ Results

?s	?h	?w
http://example.org/ontology/Sistine_Chapel		
http://example.org/ontology/David		
http://example.org/ontology/Last_Supper	4.60	8.80
http://example.org/ontology/Mona_Lisa	0.77	0.53

■ Query

```
PREFIX : <http://example.org/ontology/>

SELECT ?s ?h ?w
WHERE { ?s a :Artwork.
        OPTIONAL { ?s :heightMetric ?h . }
        OPTIONAL { ?s :widthMetric ?w . }
}
```

■ Results

?s	?h	?w
http://example.org/ontology/Sistine_Chapel		
http://example.org/ontology/David	4.86	
http://example.org/ontology/Last_Supper	4.60	8.80
http://example.org/ontology/Mona_Lisa	0.77	0.53

Matching alternatives with UNION

■ Query

```
PREFIX : <http://example.org/ontology/>

SELECT ?s
WHERE {
    { ?s a :Painting. } UNION { ?s a :Sculpture. }
}
```

■ Results

?s

http://example.org/ontology/Sistine_Chapel

http://example.org/ontology/Last_Supper

http://example.org/ontology/Mona_Lisa

http://example.org/ontology/David

Are you following? Let's check!



[source http://origin-www.yoursingapore.com/content/traveller/zh/browse/see-and-do/hands-on/_jcr_content/flash/image.img.png]

Put what illustrated at work

- For every artist tell, if it is available, the death year (Hint: <http://example.org/ontology/deathYear>)
- Find artists who were born or who died in France

Put what illustrated at work - **SOLUTION**

- For every artist tell, if it is available, the death year
(Hint: <http://example.org/ontology/deathYear>)

```
PREFIX : <http://example.org/ontology/>

SELECT ?s ?o
WHERE { ?s a :Artist .
        OPTIONAL { ?s :deathYear ?o . }
}
```

Put what illustrated at work - **SOLUTION**

- Find artists who were born or who died in France

```
PREFIX : <http://example.org/ontology/>
```

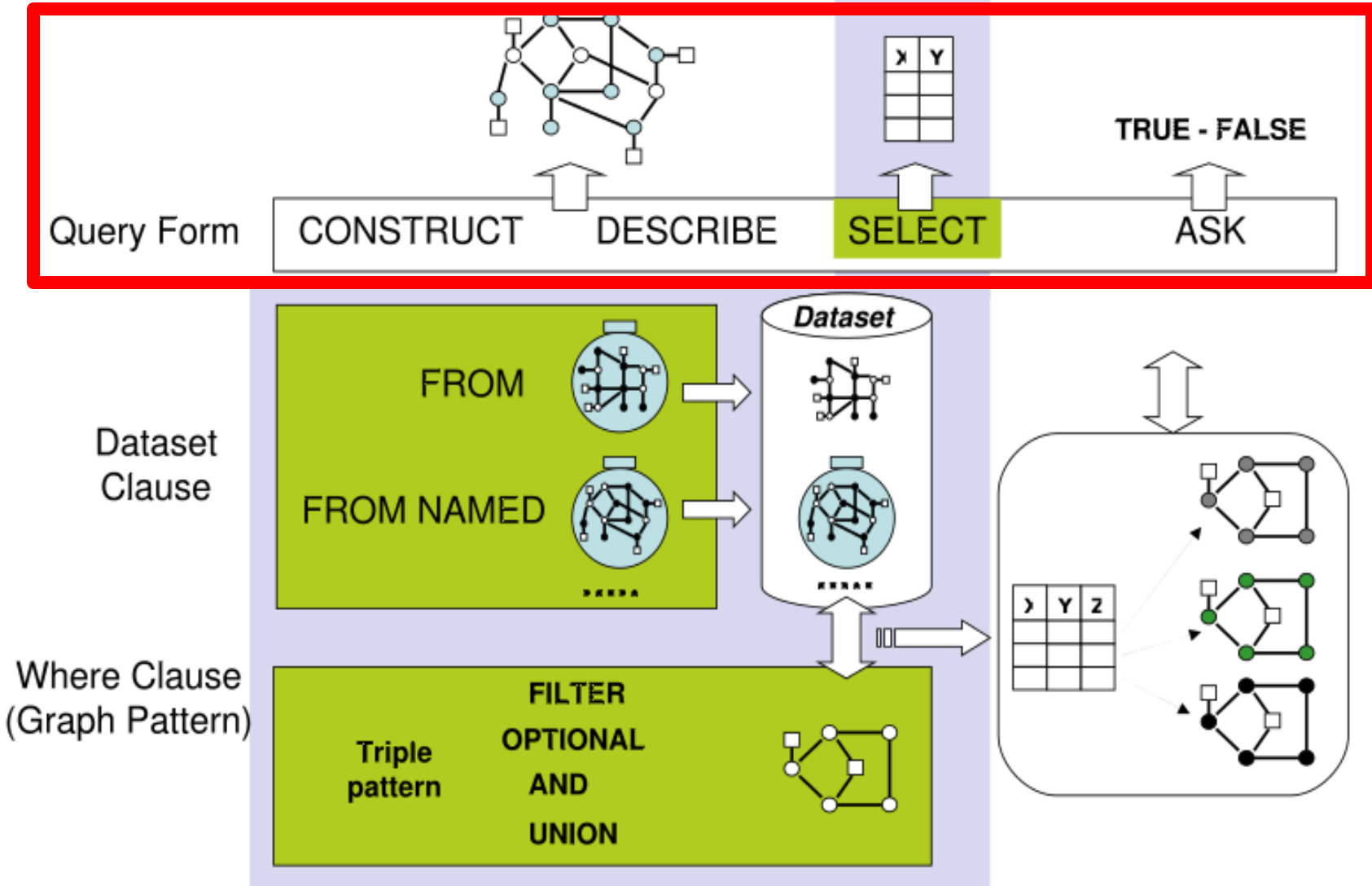
```
SELECT ?s
```

```
WHERE { ?s a :Artist .
```

```
        {?s :birthPlace :France} UNION {?s :deathPlace :France.}
```

```
}
```

Anatomy of a SPARQL query



Source: Pérez, Arenas and Gutierrez, Chapter 1: On the Semantics of SPARQL, Semantic Web Information Management: A Model Based Perspective, Springer 2010

■ Query

```
PREFIX : <http://example.org/ontology/>
```

```
SELECT ?s ?o
```

```
WHERE { ?s a :Artwork.  
        ?s :heightMetric ?o . }
```

```
ORDER BY DESC(?o)
```

■ Results

?s	?o
http://example.org/ontology/David	4.86
http://example.org/ontology/Last_Supper	4.60
http://example.org/ontology/Mona_Lisa	0.77

■ Query

```
PREFIX : <http://example.org/ontology/>
```

```
SELECT ?s ?o
```

```
WHERE { ?s a :Artwork.  
        ?s :heightMetric ?o . }
```

```
ORDER BY ASC(?o)
```

■ Results

?s	?o
http://example.org/ontology/Mona_Lisa	0.77
http://example.org/ontology/Last_Supper	4.60
http://example.org/ontology/David	4.86

Limiting the results of a SELECT query

■ Query

```
PREFIX : <http://example.org/ontology/>
```

```
SELECT ?s ?o
```

```
WHERE { ?s a :Artwork.  
        ?s :heightMetric ?o . }
```

```
ORDER BY DESC(?o)
```

```
LIMIT 2
```

■ Results

?s	?o
http://example.org/ontology/David	4.86
http://example.org/ontology/Last_Supper	4.60

Obtaining unique results

- Query

```
PREFIX : <http://example.org/ontology/>
```

```
SELECT DISTINCT ?s  
WHERE { ?s a :Artwork .  
        ?s ?p ?o . }
```

- NOTE

Without the **DISTINCT** clause, the above query returns each subject as many times as it is involved in an RDF triple

Result Forms

- Besides **SELECT**ing tables of values, SPARQL allows three other types of queries:
 - **ASK** - returns a boolean answering, does the query have any results?
 - **CONSTRUCT** - uses variable bindings to return new RDF triples
 - **DESCRIBE** - returns server-determined RDF about the queried resources
- **SELECT** and **ASK** results can be returned as XML or JSON.
- **CONSTRUCT** and **DESCRIBE** results can be returned via any RDF serialization (e.g. RDF/XML or Turtle).

Example of ASK query

- Query

```
PREFIX : <http://example.org/ontology/>
```

```
ASK
```

```
WHERE { ?s a :Artwork . }
```

- Note

- The query will return **yes** if the endpoint knows about artworks (i.e., if the pattern is matched at least once)

CONSTRUCTING results in RDF

■ Query

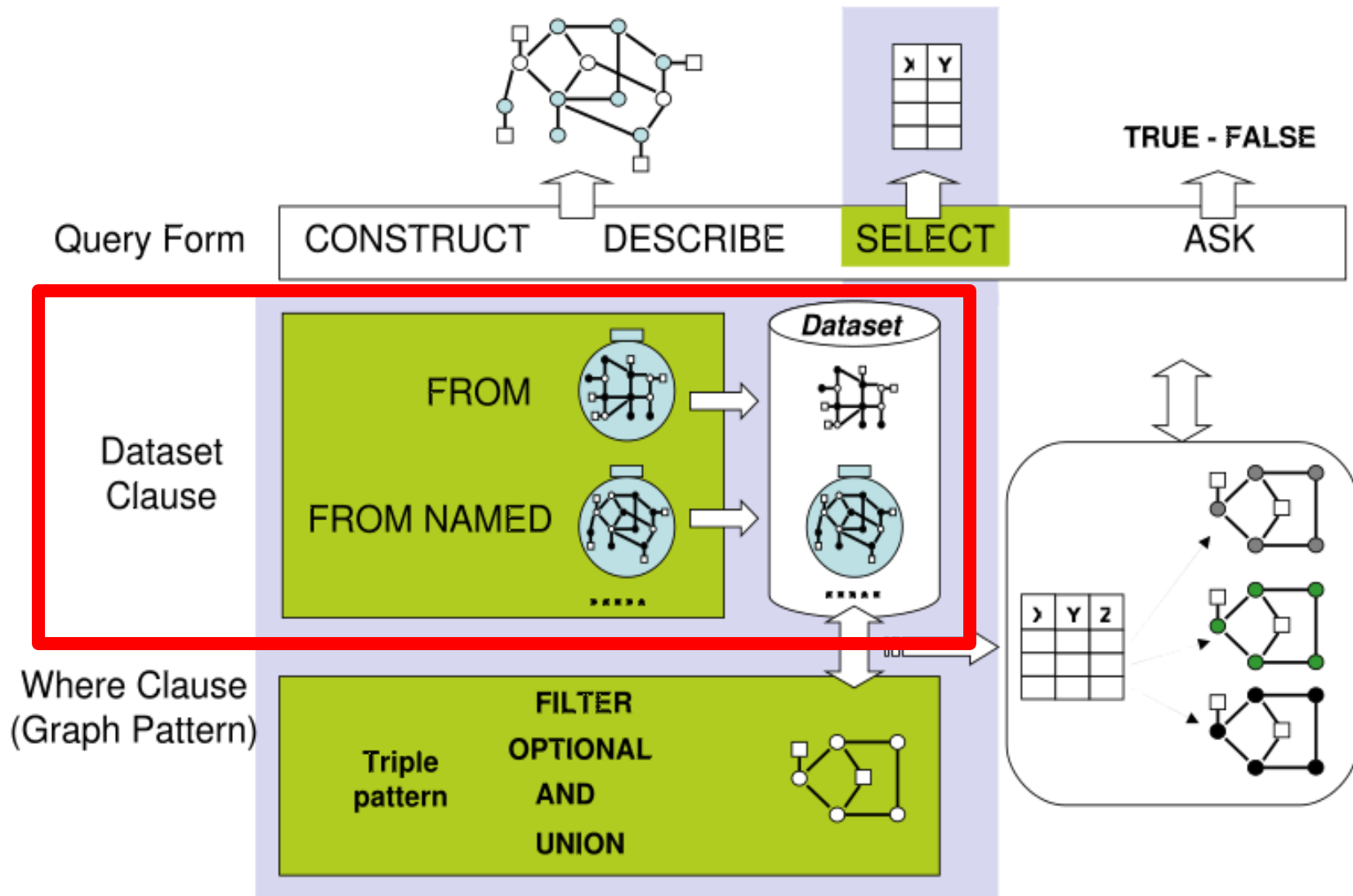
```
PREFIX :      <http://example.org/ontology/>
PREFIX rdf:   <http://www.w3.org/1999/02/22-rdf-syntax-ns#>

CONSTRUCT { ?y rdf:type :ItalianSculptor }
WHERE { ?x a :Sculpture .
        ?x :author ?y .
        ?y :birthPlace :Italy .
      }
```

■ NOTE

- The query construct a new triple for each Italian sculptor
- The new triples can be added back to the dataset reducing its variety

Anatomy of a SPARQL query



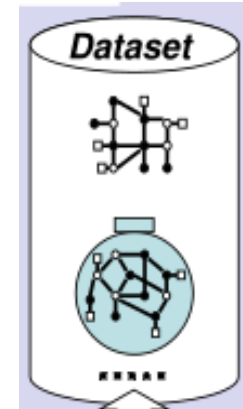
Source: Pérez, Arenas and Gutierrez, Chapter 1: On the Semantics of SPARQL, Semantic Web Information Management: A Model Based Perspective, Springer 2010

Multiple graphs and the dataset

- All the queries we have seen so far have operated on single RDF graphs.
- A SPARQL query actually runs on an **RDF dataset** which may include **multiple RDF graphs**.
- RDF graphs are **identified**, like everything else, **by URI**.
- Using multiple FROM declaration several graphs can be combined in a dataset
- Note
 - If the dataset is not in the RDF repository the SPARQL engine will try to download it from the Web
 - If it is in the dataset it will limit the scope of the query to the declared graph

Multiple graphs

- The **dataset** can be composed of
 - **one** (optional) **default graph**
 - **any number of named graphs**



- The **FROM** keyword **specifies the default graph**
 - If you use several FROM keywords, the specified graphs are merged to create the default graph
- The **FROM NAMED** keyword add **named graphs** to the dataset
- The **GRAPH** keyword is required **to match patterns in a named graph** by explicitly stating which graph they must match in.

SPARQL 1.1

- Updated 1.1 versions of SPARQL Query
- SPARQL 1.1 Basic Federated Query
- SPARQL 1.1 Update
- Updated 1.1 version of SPARQL Protocol
- SPARQL 1.1 Graph Store HTTP Protocol
- SPARQL 1.1 Service Descriptions
- SPARQL 1.1 Entailments

Aggregates

- The new built-in aggregate expressions are:
 - AVG(expr)
 - COUNT(*) and COUNT(expr)
 - GROUP_CONCAT(expr) with optional ;separator = 'string'
 - MAX(expr)
 - MIN(expr)

Sub-queries

■ Query

```
PREFIX : <http://example.org/ontology/>
```

```
SELECT ?a ?x ?y
```

```
WHERE { ?x :author ?a .
```

```
       ?x :year ?y .
```

```
       { SELECT (MIN(?y) AS ?miny)
```

```
         WHERE { ?x :year ?y . } }
```

```
       FILTER (?y = ?miny)
```

```
}
```

■ Results

?a	?x	?y
http://example.org/ontology/Michelangelo	http://example.org/ontology/Sistine_Chapel	1473

Negation

- 2 forms – stylistic and conceptual differences
 - **MINUS**: absence of a pattern
 - based on testing whether a pattern exists in the data given the bindings already determined by the query pattern
 - **FILTER NOT EXISTS**: Removing rows from a result set
 - based on removing matches based on the evaluation of two patterns
- Nearly the same, but not quite

Negation – MINUS form

■ Query

```
PREFIX : <http://example.org/ontology/>

SELECT ?x
WHERE { ?x a :Artwork .
        MINUS { ?x :author :Michelangelo . }
}
```

■ Results

?x
http://example.org/ontology/Last_Supper
http://example.org/ontology/Mona_Lisa

Negation – FILTER NOT EXISTS form

■ Query

```
PREFIX : <http://example.org/ontology/>
```

```
SELECT ?x
```

```
WHERE { ?x a :Artwork .
```

```
        FILTER NOT EXISTS { ?x :author :Michelangelo . }
```

```
}
```

■ Results

?x
http://example.org/ontology/Last_Supper
http://example.org/ontology/Mona_Lisa

Negation – Examples – Different Results

■ Data

```
@prefix : <http://example/> . :a :b :c .
```

■ Query

```
SELECT *
WHERE {
  ?s ?p ?o
  FILTER NOT EXISTS {
    ?x ?y ?z }
}
```

■ Results

0 results

because { ?x ?y ?z } matches given any ?s ?p ?o, so NOT EXISTS { ?x ?y ?z } eliminates any solutions.

```
SELECT *
WHERE {
  ?s ?p ?o
  MINUS {
    ?x ?y ?z }
}
```

?s	?p	?o
:a	:b	:c

there is no shared variable between the first part (?s ?p ?o) and the second (?x ?y ?z) so no bindings are eliminated.

Basic Federated Queries

- A graph pattern that invokes a SPARQL protocol call and remote query returning the usual result formats

```
PREFIX : <http://example.org/ontology/>
PREFIX dbo: <http://dbpedia.org/ontology/>
PREFIX owl: <http://www.w3.org/2002/07/owl/>

SELECT ?x ?yr ?abst
WHERE { ?x :year ?yr .
        ?x owl:sameAs ?dbo.
        SERVICE <https://dbpedia.org/sparql/> {
            ?dbo dbo:abstract ?abst.
            FILTER (lang(?abst) = "en")
        }
}
```

- In the RDF dataset, we used the `owl:sameAs` property to express the semantical equivalence of two objects

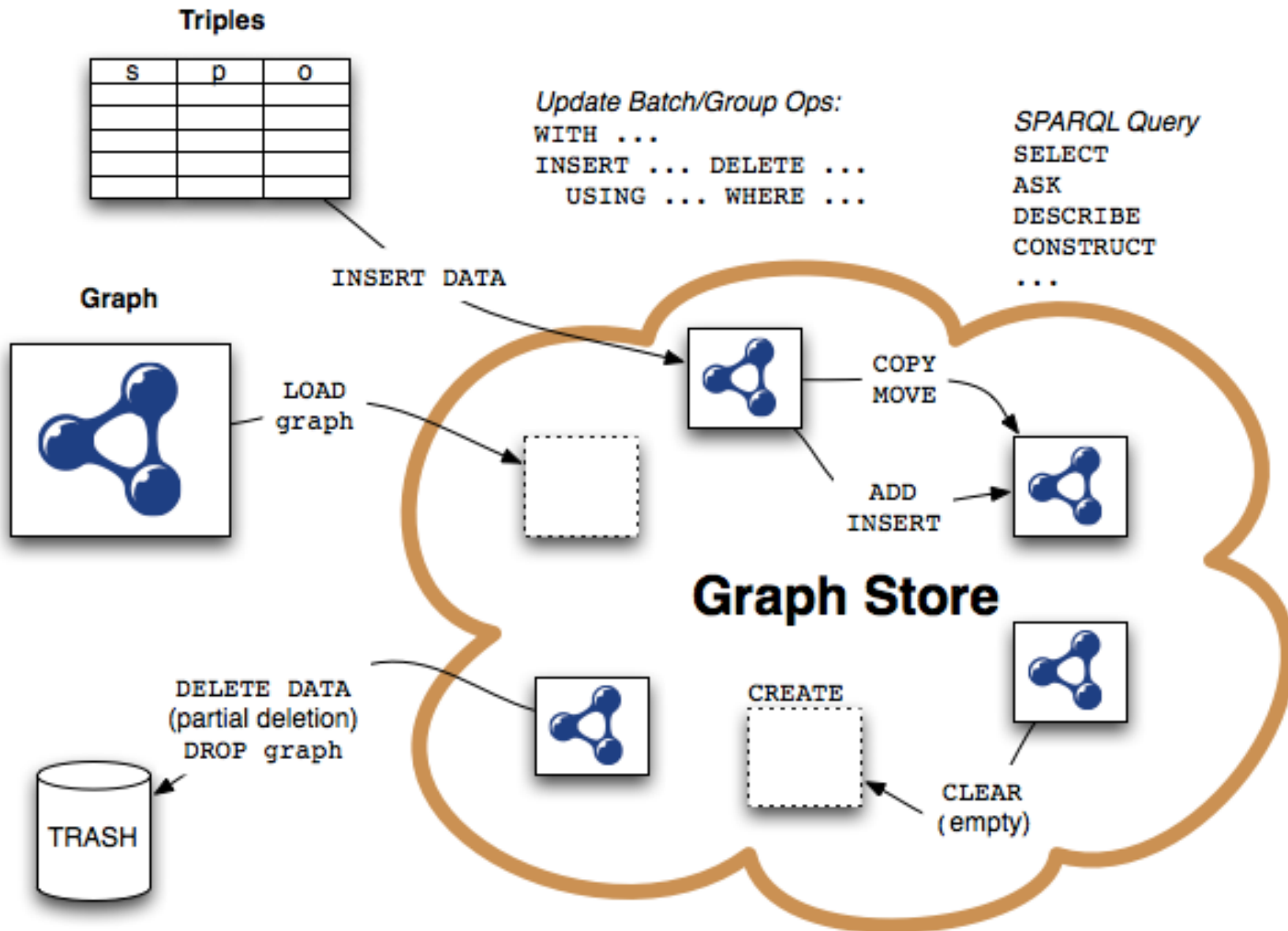
```
:Mona_Lisa owl:sameAs dbr:Mona_Lisa .
:Last_Supper owl:sameAs dbr:Last_Supper .
```

Basic Federated Queries

■ Results

?x	?yr	?abst
http://example.org/ontology/Sistine_Chapel	1473	The Sistine Chapel (/ˌsɪstɪːn ˈtʃæpəl/; Latin: Sacellum Sixtinum; Italian: Cappella Sistina [kapˈpɛlla siˈstiːna]) is a chapel in the Apostolic Palace, the official residence of the pope, in Vatican City. Originally known as the Cappella Magna ('Great Chapel'), the chapel takes its name from Pope Sixtus IV, who built it between 1473 and 1481. Since that time ...
http://example.org/ontology/David	1501	David is a masterpiece of Renaissance sculpture, created in marble between 1501 and 1504 by the Italian artist Michelangelo. David is a 5.17-metre (17 ft 0 in) marble statue of the Biblical figure David, a favoured subject in the art of Florence. David was originally commissioned ...
http://example.org/ontology/Last_Supper	1495	The Last Supper (italian: Il Cenacolo [il tʃeˈnaːkolo] or L'Ultima Cena ['lultima ˈtʃeːna]) is a late 15th-century mural painting by Italian artist Leonardo da Vinci housed by the refectory of the Convent of Santa Maria delle Grazie in Milan, Italy. It is one of the Western world's most recognizable paintings. The work is assumed to ...
...	...	...

SPARQL Update



SPARQL Resources

- SPARQL Frequently Asked Questions
 - <http://thefigtrees.net/lee/sw/sparql-faq>
- SPARQL implementations - community maintained list of open-source and commercial SPARQL engines
 - <http://esw.w3.org/topic/SparqlImplementations>
- Public SPARQL endpoints - community maintained list
 - <http://esw.w3.org/topic/SparqlEndpoints>
- SPARQL extensions - collection of SPARQL extensions implemented in various SPARQL engines
 - <http://esw.w3.org/topic/SPARQL/Extensions>

Note

- When cutting and pasting the SPARQL query from the slides into a query textbox you may get:



Response was not OK, was: 400 Can't run query: Reason: Lexical error at line 5, column 34. Encountered: "\u200b" (8203), after : ""

- A workaround is changing all the (so called) white spaces:
 - e.g., in MS Word replace "^w" with " "

Acknowledgments

- Some slides in this presentation were adapted from
www.dcs.warwick.ac.uk/~acristea/courses/CS411/2010/SPARQL.ppt
<http://www.emse.fr/~zimmermann/Teaching/WebSem/SlidesSPARQL.html>